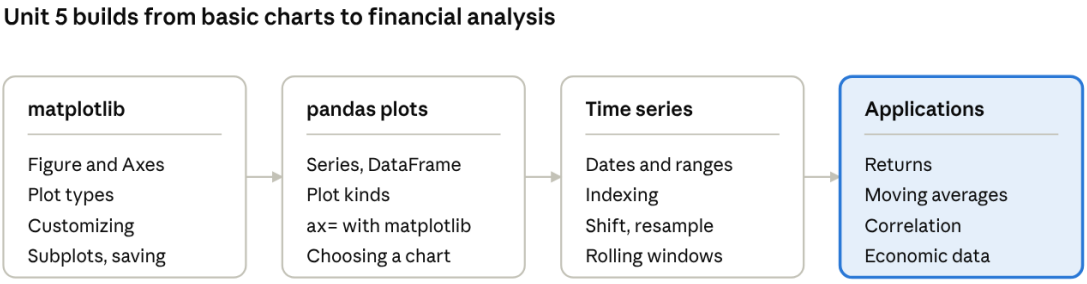
Data Science Unit 5: Data Visualization

Unit overview

Part	Syllabus topics	What you produce
matplotlib	A brief matplotlib API primer	Line, bar, scatter and histogram charts with titles, labels and legends
pandas plotting	Plotting functions in pandas	Charts straight from a Series or DataFrame in one line
Time series	Dates, indexing, resampling, shifting, rolling windows	Trends over time and smoothed or re-sampled views
Applications	Financial and economic data	Returns, moving averages and correlation of prices and indicators



unit roadmap · 4 parts

1. A Brief matplotlib API Primer

matplotlib is the foundation of Python plotting: you create a **Figure**, draw on one or more **Axes** inside it, and then style and save the result.

1.1 Figure, Axes and the two interfaces

A **Figure** is the whole canvas (the window or image). An **Axes** is one plot area inside it, with its own x-axis, y-axis, title and labels. One Figure can hold many Axes.

matplotlib has two ways to draw. The **pyplot** (state-based) style calls `plt.plot(...)` and works on whichever Axes is current, which is quick for experiments. The **object-oriented** style keeps explicit `fig` and `ax` objects, which is clearer and is preferred for anything with more than one plot.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0, 10, 100)

fig, ax = plt.subplots(figsize=(6, 4))          # one Figure holding one Axes
ax.plot(x, np.sin(x), label="sin(x)", color="tab:blue", linestyle="-")
ax.plot(x, np.cos(x), label="cos(x)", color="tab:orange", linestyle="--")

ax.set_title("Sine and cosine")
ax.set_xlabel("x")
ax.set_ylabel("value")
ax.legend()                                      # uses the label= text of
each line
ax.grid(True)

fig.savefig("waves.png", dpi=150)                # save to a file
plt.show()                                       # display the window
```

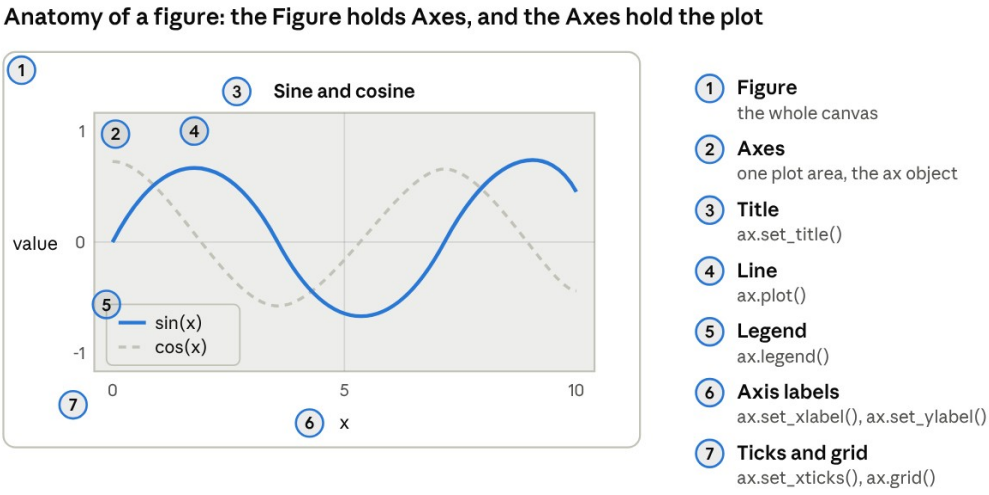


figure anatomy · 7 labelled parts

The numbers on the picture match the numbered list: each part of the chart is created or changed by one method of the `ax` object.

1.2 Plot types

Method	Chart	Use it to show
`ax.plot(x, y)`	Line	A trend, usually over time
`ax.scatter(x, y)`	Scatter	The relationship between two numeric variables
`ax.bar(names, heights)` / `ax.barh`	Bars (vertical or horizontal)	Comparing categories
`ax.hist(values, bins=10)`	Histogram	The distribution of one variable
`ax.boxplot(data)`	Box plot	Median, quartiles and outliers

Method	Chart	Use it to show
		of groups
`ax.pie(sizes, labels=...)`	Pie	Parts of a whole, for a few categories only
`ax.errorbar(x, y, yerr=e)`	Line with error bars	Uncertainty in measurements
`ax.fill_between(x, y1, y2)`	Shaded band	A range or confidence interval
`ax.imshow(matrix)`	Image or heat map	A grid of values such as a correlation matrix

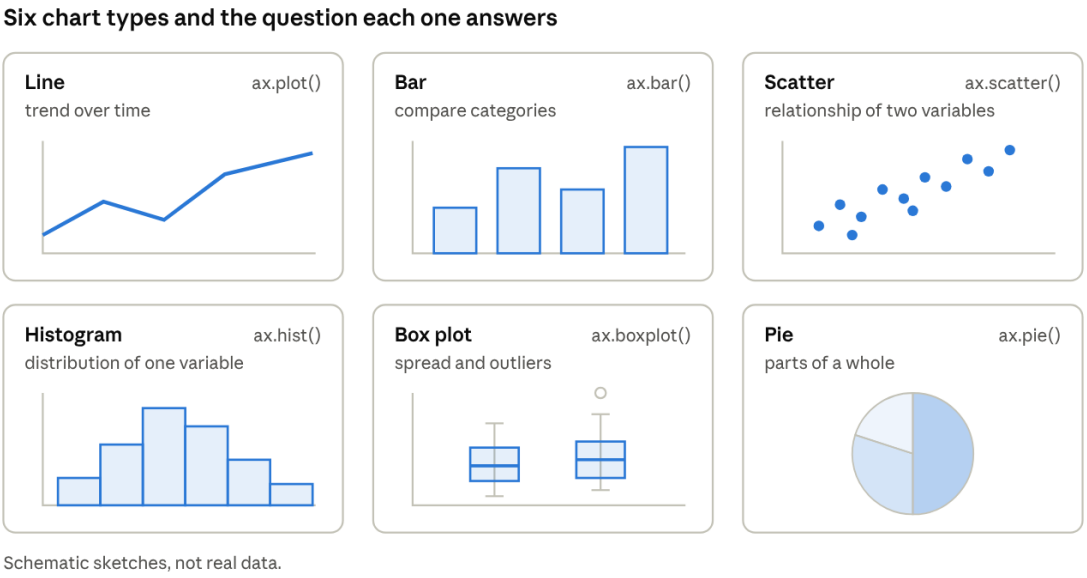


chart type gallery · 6 schematic sketches

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12, 3.5))

ax1.bar(["CSE", "IT", "ECE"], [72, 65, 58], color="tab:blue")           #
bars
ax1.set_title("Average marks")

rng = np.random.default_rng(1)
ax2.scatter(rng.normal(size=50), rng.normal(size=50), alpha=0.6)         #
scatter
ax2.set_title("Scatter")

ax3.hist(rng.normal(size=500), bins=20, color="tab:green")               #
histogram
ax3.set_title("Histogram")
```

```
fig.tight_layout()
plt.show()
```

1.3 Customizing a chart

- **Lines and markers:** `color`, `linestyle` (`"-"`, `"--"`, `":"`), `linewidth`, `marker` (`"o"`, `"s"`, `"^"`) and `markersize`. A format string combines them: `ax.plot(x, y, "ro--")` means red circles joined by a dashed line.
- **Text:** `set_title`, `set_xlabel`, `set_ylabel`, `ax.text(x, y, "note")` and `ax.annotate("peak", xy=(5, 1), xytext=(6, 1.2), arrowprops=dict(arrowstyle="->"))`.
- **Limits and ticks:** `set_xlim(0, 10)`, `set_ylim(-1, 1)`, `set_xticks([0, 5, 10])`, `set_xticklabels(["low", "mid", "high"])`.
- **Legend:** `ax.legend(loc="upper right")`; every plotted item needs a `label=`.
- **Appearance:** `ax.grid(True)`, `plt.style.use("ggplot")` for a ready-made theme, and `fontsize=` on any text.
- **Colours:** names (`"tab:red"`), single letters (`"r"`) or hex codes (`"#2a7ae2"`); `alpha` sets transparency.

1.4 Subplots

`plt.subplots(nrows, ncols)` returns the Figure and an array of Axes. With one row or column the array is one-dimensional, otherwise it is two-dimensional.

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6), sharex=True)   # 2 x 2 grid
sharing the x-axis

axes[0, 0].plot(x, np.sin(x));  axes[0, 0].set_title("sin")
axes[0, 1].plot(x, np.cos(x));  axes[0, 1].set_title("cos")
axes[1, 0].plot(x, x);          axes[1, 0].set_title("line")
axes[1, 1].plot(x, x ** 2);     axes[1, 1].set_title("square")

fig.tight_layout()            # stop titles and labels overlapping
```

1.5 Saving figures

`fig.savefig("chart.png", dpi=300, bbox_inches="tight")` writes an image; the extension chooses the format (`.png`, `.pdf`, `.svg`, `.jpg`). Use PDF or SVG when the chart must stay sharp at any size, and PNG for web pages. In a Jupyter notebook, charts appear under the cell automatically.

2. Plotting Functions in pandas

Series and DataFrame have a built-in `plot` method that uses matplotlib underneath, so one line of code gives a labelled chart and the index and column names become the axis and legend automatically.

2.1 Series and DataFrame plots

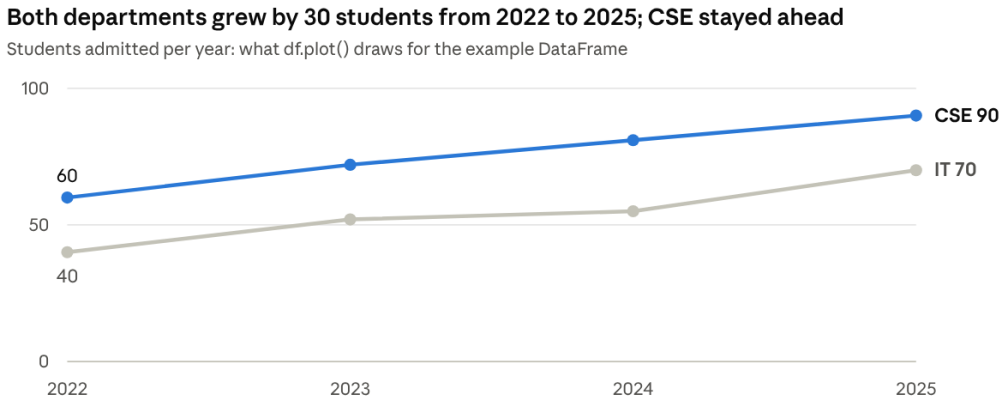
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "year": [2022, 2023, 2024, 2025],
```

```
    "CSE":  [60, 72, 81, 90],
    "IT":   [40, 52, 55, 70],
}).set_index("year")

df.plot()                                   # one line per column; the index
is the x-axis
df.plot(kind="bar")                         # grouped bars
df.plot(kind="bar", stacked=True)           # stacked bars
df["CSE"].plot(title="CSE admissions", color="tab:blue", marker="o")

plt.show()
```



example DataFrame from section 2.1 · 2022 to 2025

The index (years) becomes the x-axis and each column becomes one line with its own legend entry, with no extra code.

The `kind` argument chooses the chart; each kind also has a method form such as `df.plot.bar()`.

`kind`	Chart	Typical use
`"line"` (default)	Line plot	Trends over time
`"bar"`, `"barh"`	Vertical or horizontal bars	Comparing categories; add `stacked=True` for parts of a total
`"hist"`	Histogram	Distribution of one column
`"kde"` or `"density"`	Smooth density curve	Shape of a distribution
`"box"`	Box plot	Spread and outliers per column
`"scatter"`	Scatter (needs `x=` and `y=`)	Relationship between two columns
`"area"`	Filled area	Cumulative totals over time
`"pie"`	Pie chart	Shares of a whole

2.2 Useful plot options

- `title`, `xlabel`, `ylabel`, `legend=False`, `grid=True`
- `figsize=(8, 4)` for the size, `rot=0` to rotate tick labels, `alpha=0.6` for transparency
- `subplots=True` draws each column in its own panel, with `layout=(2, 2)` to arrange them
- `logy=True` for a logarithmic y-axis, `xlim` and `ylim` to set the range, `colormap` for a colour scheme

```
# NumPy supplies random sample data for this example
import numpy as np
rng = np.random.default_rng(0)

data = pd.DataFrame({
    "height": rng.normal(165, 8, 200),
    "weight": rng.normal(62, 9, 200),
})

data["height"].plot.hist(bins=15, alpha=0.7, title="Height distribution")
data.plot.scatter(x="height", y="weight", alpha=0.5)      # relationship of
two columns
data.plot.box()                                           # one box per
column
pd.plotting.scatter_matrix(data, figsize=(5, 5))          # all pairs of
columns
```

2.3 Combining pandas with matplotlib

Every pandas plot method returns a matplotlib `Axes`, and accepts one through `ax=`. That lets you place several pandas charts on one Figure and finish the styling with matplotlib.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10, 4))

df.plot(ax=left, title="Lines")
df.plot.bar(ax=right, title="Bars")
right.set_ylabel("Students")

fig.tight_layout()
fig.savefig("admissions.png", dpi=150)
```

The seaborn library builds on matplotlib and pandas to give statistical charts such as heat maps and pair plots with attractive defaults; it works directly with DataFrames.

2.4 Choosing the right chart

Your goal	Good choice
Show change over time	Line plot
Compare values across categories	Bar chart (horizontal when the names are long)
Show how values are distributed	Histogram, density plot or box plot
Show the relationship between two numeric columns	Scatter plot, with a scatter matrix for many columns

Your goal	Good choice
Show parts of a whole	Stacked bar (a pie only for very few parts)
Show a grid of values such as correlations	Heat map (`imshow` or seaborn)

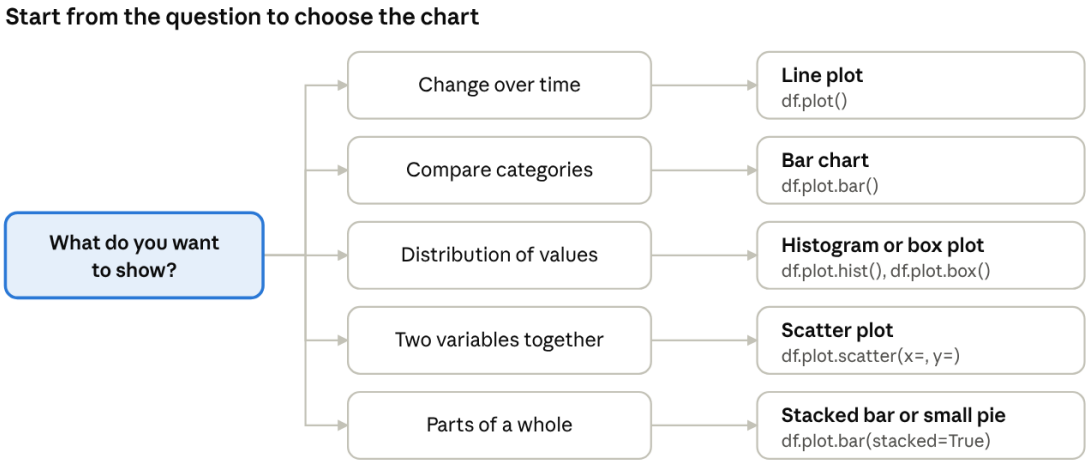


chart chooser · 5 questions

3. Time Series

A time series is data indexed by time; pandas stores it with a `DatetimeIndex`, which makes selecting periods, changing the frequency and computing moving averages one-line operations.

3.1 Date and time types and date ranges

Python's `datetime` module has `date`, `time` and `datetime` objects; pandas adds `Timestamp` (one moment), `Timedelta` (a duration) and `Period` (a span such as a month). `pd.to_datetime` converts text to timestamps, and `pd.date_range` builds a regular sequence of dates.

```
import numpy as np
import pandas as pd

print(pd.to_datetime("2026-03-05"))                          #
Timestamp('2026-03-05 00:00:00')
print(pd.to_datetime(["05/03/2026"], dayfirst=True))          # day before
month

rng = pd.date_range("2026-01-01", periods=90, freq="D")       # 90 daily dates
ts = pd.Series(np.arange(90, dtype=float), index=rng)
print(ts.head(3))
# 2026-01-01    0.0
# 2026-01-02    1.0
# 2026-01-03    2.0
```

Frequency code	Meaning
`"D"`	Calendar day

Frequency code	Meaning
`"B"`	Business day (Monday to Friday)
`"W"`	Week (ending on Sunday)
`"MS"`	Month start
`"ME"`	Month end (older pandas versions write `"M"`)
`"h"`, `"min"`	Hour, minute

3.2 Time series indexing and selection

With a `DatetimeIndex` you can select by date strings, and a partial string such as a month selects the whole period.

```
print(ts["2026-02-10"])                 # one day: 40.0
print(ts["2026-03"].head(2))            # every day of March
print(ts["2026-02-01":"2026-02-05"])    # a slice; both ends are included
print(ts.index.month[:3])               # parts of the date are available:
Index([1, 1, 1])
```

3.3 Shifting and percentage change

`shift(n)` moves the **data** forward (positive) or backward (negative) by `n` positions while the index stays put, which is how you compare a value with the previous one. `pct_change()` is the relative change from the previous value.

```
print(ts.shift(1).head(3).tolist())      # [nan, 0.0, 1.0]
print(ts.shift(-1).head(3).tolist())     # [1.0, 2.0, 3.0]
print(ts.shift(7, freq="D").index[0])    # with freq the INDEX moves: 2026-
01-08

price = pd.Series([100.0, 102.0, 101.0], index=pd.date_range("2026-01-01",
periods=3))
print(price.pct_change().round(4).tolist())   # [nan, 0.02, -0.0098]
```

3.4 Time zones

Timestamps are *naive* (no zone) until you attach one. `tz_localize` attaches a zone and `tz_convert` changes it.

```
stamp = pd.Timestamp("2026-03-05 09:30")
local = stamp.tz_localize("Asia/Kolkata")        # attach a time zone
print(local.tz_convert("UTC"))                   # 2026-03-05 04:00:00+00:00
```

3.5 Resampling and moving windows

Resampling changes the frequency of a series. *Downsampling* (daily to monthly) needs an aggregation such as `mean` or `sum`; *upsampling* (monthly to daily) creates gaps that you fill with `ffill` or interpolation.

Moving window functions compute a statistic over a sliding set of the most recent values, which smooths noise: `rolling(window).mean()` is a moving average, `rolling(window).std()` a moving volatility, and `expanding()` uses everything so far.

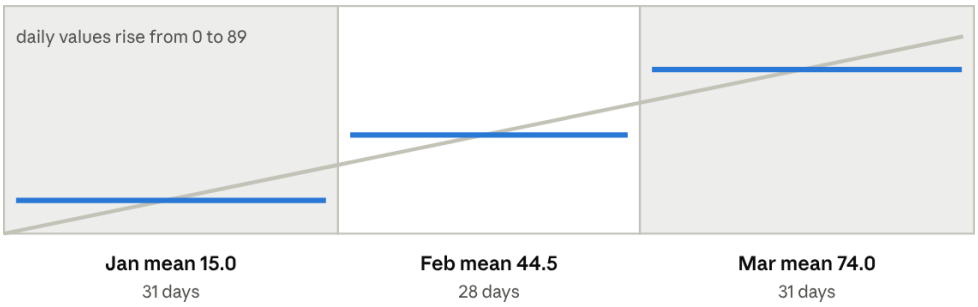
```
monthly = ts.resample("MS").mean()
print(monthly.tolist())                    # [15.0, 44.5, 74.0]   January,
February, March

print(ts.resample("W").sum().head(2))      # weekly totals

ma7 = ts.rolling(7).mean()                 # 7-day moving average
print(ma7.iloc[5], ma7.iloc[6])            # nan 3.0   the first full window
ends on day 7

print(ts.expanding().mean().iloc[3])       # 1.5   mean of the first four
values
```

resample('MS').mean() turns 90 daily values into 3 monthly averages

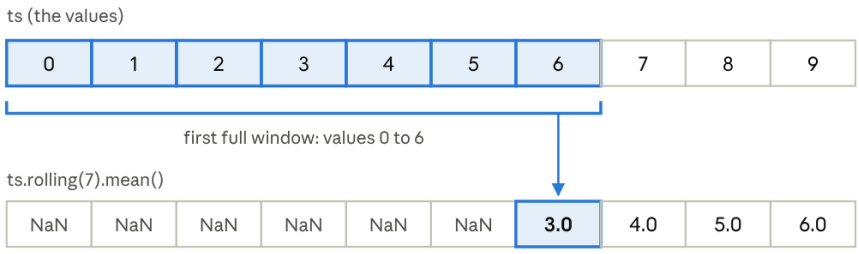


monthly averages from the 90-day example

Each monthly average sits halfway up its band, because the daily values rise steadily.

The first `window - 1` results of a rolling calculation are `NaN` because there are not yet enough values to fill a window; set `min_periods` to allow shorter windows.

A rolling window averages the latest 7 values, so the first 6 results are NaN



The window then slides one step at a time: the next result, 4.0, is the mean of values 1 to 7.

7-day rolling mean · values 0 to 9

4. Financial and Economic Data Applications

Financial and economic data are time series, so the whole unit comes together here: load prices or indicators, convert them to returns, smooth them with moving windows, compare series and chart the result.



finance workflow · 5 steps

4.1 Getting the data

Market and economic series usually arrive as a CSV file or from a data provider. A CSV with a date column loads straight into a DatetimeIndex.

```
prices = pd.read_csv("prices.csv", index_col="Date", parse_dates=True)
prices = prices.sort_index()
```

Libraries such as `pandas-datareader` and `yfinance` can download prices and public economic indicators (for example from the World Bank or FRED), but their availability and terms change, so check each library's current documentation. The examples below create their own sample prices so that they run anywhere.

4.2 Returns, moving averages and volatility

Investors look at **returns** (the percentage change) rather than raw prices because returns are comparable across assets.

```
r_t = \frac{P_t - P_{t-1}}{P_{t-1}}
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
dates = pd.bdate_range("2025-01-01", periods=250)          # business days
only
price = pd.Series(100 + rng.normal(0, 1, 250).cumsum(), index=dates,
name="PRICE")   # sample prices

ret = price.pct_change()                       # daily return r_t
log_ret = np.log(price / price.shift(1))     # log return, useful because it
adds up over time
cum = (1 + ret).cumprod() - 1                # cumulative return since the
```

```
start

ma20 = price.rolling(20).mean()              # 20-day moving average (trend)
ma50 = price.rolling(50).mean()              # 50-day moving average (slower
trend)
vol20 = ret.rolling(20).std()                # 20-day volatility (risk)

fig, ax = plt.subplots(figsize=(9, 4))
price.plot(ax=ax, label="Price", alpha=0.6)
ma20.plot(ax=ax, label="20-day average")
ma50.plot(ax=ax, label="50-day average")
ax.set_title("Price with moving averages (sample data)")
ax.legend()
plt.show()
```

Measure	What it tells you	pandas
Daily return	Percentage change from the previous day	`price.pct_change()`
Cumulative return	Total gain since the start	`(1 + ret).cumprod() - 1`
Moving average	Smoothed trend; a short average crossing above a long one is a common signal	`price.rolling(20).mean()`
Volatility	How widely returns vary, a measure of risk	`ret.rolling(20).std()`
Drawdown	Fall from the highest price so far	`price / price.cummax() - 1`

4.3 Resampling and comparing several series

```
monthly_close = price.resample("ME").last()            # last price of each
month ("M" in older pandas)
monthly_ret = monthly_close.pct_change()

# several assets: correlation of their returns
rets = pd.DataFrame({
    "A": price.pct_change(),
    "B": price.pct_change() * 0.5 + rng.normal(0, 0.005, 250),   # a related
sample series
}).dropna()

print(rets.corr())                                      # correlation matrix:
values from -1 to 1
print(rets["A"].rolling(60).corr(rets["B"]).tail(1))     # how the
relationship changes over time

indexed = price / price.iloc[0] * 100                   # rebase to 100 to
compare assets on one chart
```

Correlation runs from -1 (move in opposite directions) through 0 (unrelated) to +1 (move together); it is the basis of diversification, because assets that do not move together reduce risk.

4.4 Economic indicators

Economic series such as GDP, unemployment and consumer prices are published monthly, quarterly or yearly, so they often have different frequencies. The same tools apply.

- **Inflation rate:** the 12-month change of a price index, `cpi.pct_change(12) * 100` for monthly data.
- **Growth rate:** `gdp.pct_change()` for the change from the previous period.
- **Aligning frequencies:** `resample` a daily series to monthly, or use `ffill` to carry a quarterly value across its months, then join the series on the date index.
- **Smoothing:** a rolling mean removes month-to-month noise to show the trend.
- **Indexing to a base year:** divide by the base value and multiply by 100 so different indicators start at 100.

```
# monthly consumer price index (sample numbers) -> year-on-year inflation
cpi = pd.Series(np.linspace(100, 108, 24), index=pd.date_range("2024-01-01",
periods=24, freq="MS"))
inflation = cpi.pct_change(12) * 100          # percent change versus the
same month a year earlier
print(inflation.dropna().round(2).head(2))
```

Prices and indicators in this guide are made-up sample data used only to demonstrate the code; they do not describe any real market.

Quick revision

Remember one core idea for each topic.

- **Figure and Axes:** a Figure is the canvas; an Axes is one plot area with its own axes, title and labels.
- **Two interfaces:** pyplot (`plt.plot`) works on the current Axes; the object-oriented style uses `fig, ax = plt.subplots()`.
- **Basic plots:** `plot` for lines, `scatter`, `bar`, `hist`, `boxplot`, `pie`; each item needs a `label=` for the legend.
- **Customizing:** color, linestyle, marker, titles and labels, `xlim`, `ylim`, ticks, `legend`, `annotate`, `grid`, styles.
- **Subplots and saving:** `plt.subplots(rows, cols)`, `tight_layout()`, `savefig(dpi=...)` with PNG, PDF or SVG.
- **pandas plots:** `df.plot(kind=...)` with the index on the x-axis; it returns an Axes and accepts `ax=`.
- **Chart choice:** line for trends, bar for categories, histogram or box plot for distributions, scatter for relationships.
- **Time series basics:** a `DatetimeIndex`, `date_range` with a frequency code, partial string indexing such as `ts["2026-03"]`.
- **Shifting:** `shift(n)` moves the data but not the index; `pct_change()` gives the relative change.
- **Resampling:** `resample("MS").mean()` changes frequency; `rolling(n).mean()` is a moving average.

- **Finance:** returns are `pct_change`, cumulative return is `(1 + r).cumprod() - 1`, volatility is a rolling standard deviation, `corr()` compares assets.
- **Economic data:** inflation as a 12-month `pct_change`; align different frequencies with `resample` and `ffill`.

Likely exam questions

Short answer (2 to 3 marks)

1. What are a Figure and an Axes in matplotlib?
2. Differentiate between the pyplot and the object-oriented interfaces.
3. How do you create a 2 by 2 grid of subplots?
4. Name four kinds of plot available through `df.plot(kind=...)`.
5. What does `shift()` do on a time series?
6. What is the difference between resampling and rolling?
7. What is a moving average and why is it used?

Long answer (5 to 10 marks)

1. Explain the main matplotlib plot types and how a chart is customised, with a program.
2. Explain the plotting functions available in pandas, with examples.
3. Explain how time series data is created, indexed and selected in pandas.
4. Explain resampling, shifting and moving window functions, with examples.
5. Explain how stock price data is analysed in pandas using returns, moving averages and correlation.
6. Describe how economic indicators with different frequencies can be aligned and analysed.

References

Text books

1. Mark Lutz, *Learning Python*, O'Reilly.
2. Mark Lutz, *Programming Python*, O'Reilly.
3. Wes McKinney, *Python for Data Analysis*, O'Reilly (matplotlib, plotting with pandas, time series, and financial and economic data applications).

Reference books

1. Martin C. Brown, *Python: The Complete Reference*, McGraw Hill Education.
2. Paul Barry, *Head First Python*, O'Reilly.